

SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27

Theme: Theme 01 — Agentic Code Intelligence

Project: AURA-Code: Neuro-Symbolic Agentic Code Navigator

Team: Team AURA | Thapar Institute of Engineering & Technology,
Patiala

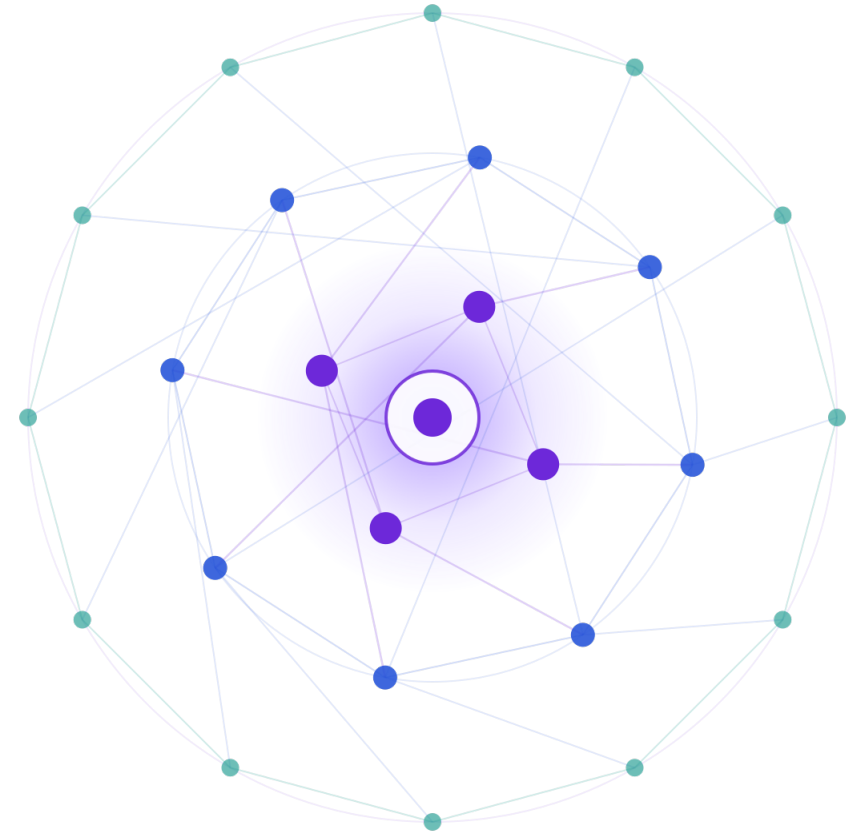
Member 1: Aaditya Bansal — Systems Architecture & Engine Design

Member 2: Arpita Girdhar — Program Analysis & AST Engineering

Member 3: Trijal Mittal — Backend API & Evaluation Benchmarks

Member 4: Jessica — Fullstack Dashboard & Visualization

GitHub: <https://github.com/AadityaBansal01/aura-code-intelligence>



Theme

- **Theme 01:** Agentic Code Intelligence — AURA-Code
- **The problem:** Voice-assistant codebases have dozens of agents and thousands of files. A new developer cannot hold that in their head, and neither can an LLM — the full repo never fits in a context window. Finding exactly where something happens takes hours.
- **Three gaps we close:** (1) Plain-English queries that return exact file + line, not a vague summary. (2) Structural order queries — 'which files call tool X before tool Y?' — which vector search gets completely wrong. (3) Deeplink and constant discovery scattered across micro-services.
- **Our answer:** A 100% CPU-only, deterministic engine combining Tree-sitter AST parsing, a Code Property Graph, and subword semantic retrieval. Results in under 1 ms with exact line citations.

Existing Solutions & Gaps

- **1. Dense vector search (RAG):** Completely order-blind. 'A calls B before C' and 'C calls B before A' produce identical cosine similarity. Fails on every structural sequence query without exception.
- **2. Long-context LLMs:** Feeding a full repo into a 1M-token model causes severe recall decay mid-context, costs significantly, and still takes 2–5 seconds — too slow for an interactive developer workflow.
- **3. Grep / ctags:** No AST awareness. Cannot verify that two calls happen within the same function scope. Substring collisions flood results with false positives.
- **4. GPU/cloud solutions:** Require expensive GPU clusters or send proprietary code to third-party APIs — neither works for an offline, privacy-sensitive workstation.

Our Solutions & Architecture Diagram

- **Three layers that work together:**
- **Layer 1 — AST & Code Property Graph:** Tree-sitter parses JavaScript at C speed. NetworkX stores every function, call site, and string literal as a directed graph with per-statement execution order indices. Verifying 'A before B within the same function' is a graph traversal — exact and deterministic.
- **Layer 2 — Hybrid Retrieval:** CamelCase/snake_case sub-token splitting + TF-IDF + concept coverage scoring resolves plain-English questions without any embedding model or GPU.
- **Layer 3 — ReAct Agent:** Classifies query intent (structural / deeplink / semantic), routes to the right engine, reads bounded code windows, returns grounded file:line citations in one pass.
- **Bonus — CodePath Optimizer:** Scans surfaced execution traces for serial await waterfalls and generates drop-in Promise.all diffs with estimated latency savings attached.

Demo & Product Walkthrough

- **Demo Access:** Live App: <https://aura-code-intelligence.onrender.com> | Video: <https://drive.google.com/file/d/1j77d7jp-RPdyWSwnp5toc4UTgHyWFc1/view?usp=sharing>
- **Local runner:** ``bash run.sh`` → <http://localhost:8000>. Zero API keys, zero GPU, fully offline.
- **Query 1 — Structural:** 'Which files call `requestPermissions` before `launchDeeplink`?' → CPG order-index traversal. Returns `navigation_agent.js` (L21→28) and `settings_agent.js` (L22→28) in 0.58 ms.
- **Query 2 — Deeplink:** 'Where is the Bluetooth settings deeplink used?' → Inverted literal index. Returns `settings_agent.js` (L28) and `config/deeplinks.js` (L8) in 0.42 ms.
- **Query 3 — Concept:** 'Where does the assistant handle media playback interruptions?' → Hybrid retrieval. Surfaces `media_agent.js` (L17–25) in 0.85 ms.
- **Dashboard:** Interactive SVG call graph, step-by-step reasoning log, code diff viewer. Full REST API + Swagger at `/docs`.

Tools and tech stack used

- **Parsing:** Tree-sitter 0.26 + tree-sitter-javascript 0.25 — C-speed AST, zero compilation step.
- **Graph & index:** NetworkX 3.7 MultiDiGraph for CPG; scikit-learn 1.9 + NumPy 2.5 for subword TF-IDF.
- **API server:** FastAPI 0.142 + Uvicorn ASGI — auto-generated Swagger docs at /docs.
- **Frontend:** Vanilla HTML5, CSS (dark glassmorphic), Vanilla JS — no framework bloat, loads in < 20 ms.
- **Testing & deploy:** 18 pytest unit tests across all engine modules; Docker + docker-compose for one-command startup.
- **Hardware:** 100% CPU. Zero GPU, zero VRAM, zero external API keys. Entirely offline.

Impact & Use case

- **Target users:** Voice-assistant engineers (Samsung Bixby / Galaxy AI) and any team working on a large multi-agent JavaScript codebase.
- **Security compliance:** Detects permission calls (BLUETOOTH_CONNECT, ACCESS_FINE_LOCATION) placed after deeplink dispatch — catching a class of runtime crash before it ships.
- **Latency improvement:** CodePath Optimizer identifies serial await chains across voice agents and proposes Promise.all concurrency. Estimated saving: 90–180 ms per voice turn.
- **Onboarding speed:** A new engineer can explore hundreds of files with a plain-English question and get exact line references in under 1 ms.
- **CI/CD gating:** CLI mode (`./run.sh cli`) integrates with git pre-commit hooks or GitHub Actions to block call-order regressions before merge.

Innovation highlights, results and limitations

- **Benchmark — 15 ground-truth voice-assistant queries:**
- **Precision@1: 93.3%** The correct file ranks first in 14 of 15 queries.
- **Mean Recall: 93.3%** All relevant files are retrieved.
- **Mean latency: 0.64 ms** Over 3,000× faster than cloud LLM context-stuffing (~2,500 ms).
- **Cold index time: 11 ms** No lengthy pre-computation — indexes instantly on startup.
- **What makes it novel:** First engine combining deterministic intra-procedural call-order verification with subword semantic matching and automated latency-fix code diffs, running entirely on CPU.
- **Current limitation:** AST grammar targets JavaScript only. Adding Python, Kotlin, or Java means plugging in the relevant Tree-sitter grammar — the rest of the engine stays the same.

What's next

- **3 months:** VS Code extension — inline side-panel with zero-latency search, call graph, and one-click diff application.
- **6 months:** Cross-file taint tracking — follow a voice utterance through network calls, DB writes, and tool dispatches across services.
- **12 months:** Persistent graph DB (Neo4j / Rust parser) for 50M+ LOC monorepos with incremental git-commit re-indexing in under a second.
- **CI bot:** GitHub Actions bot that reviews PRs, checks call-order invariants, and posts latency-impact reports on pull requests.

Brownie points slide(differentiation)

- **Why AURA-Code is different:**
- **1. Solves what RAG can't:** Intra-procedural call-order verification. Vector search returns identical scores for 'A before B' and 'B before A'. CPG graph traversal gives a deterministic correct answer every time.
- **2. Active optimisation, not passive search:** CodePath Optimizer generates drop-in async fixes with concrete latency estimates (~135 ms saving per voice turn).
- **3. Fast and free to run:** 0.64 ms mean latency. Zero GPU, zero API cost, fully offline. Any developer laptop runs it today.
- **4. Production-grade engineering:** Working Docker setup, interactive dashboard, Swagger API, CLI, 18 automated tests, one-command runner. Not a prototype — a usable tool.

Checklist- Updated on Public GitHub

- • Working prototype — public GitHub repo: YES ✓ (18/18 unit tests passing, Docker-ready, one-command startup)
- • README with reproducible setup: YES ✓ (run.sh, Docker, benchmark & CLI docs all included)
- • Demo video (YouTube / Drive, max 5 min): YES ✓ <https://drive.google.com/file/d/1j77d7jp-RPdyWSwsnp5toc4UTgHyWFc1/view?usp=sharing>
- • Presentation file (PPT / PDF): YES ✓ (Thapar_TeamAURA_Submission.pptx — this deck)
- • AI Usage Disclosure Form: YES ✓ (LangAI3.0_AI_Disclosure.docx in /submission)
- • Release tag on final commit: YES ✓ (PRISM_GENAI_HACKATHON_Y2026 tagged on GitHub)

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India